Technical Design Document (TDD)

Chikkamagaluru Estate Resort Direct-Booking Website & AI Enquiry Triage Platform

Client: Company A, Chikkamagaluru, Karnataka

Candidate Submission: Software Engineering Intern Take-Home Task (IsoCode Labs)

Submission Deadline: 9 Nov 2026, 11:59 pm IST

- **GitHub Repository**: https://github.com/PrakashMannam/chikkamagaluru-resort-booking
- **Live Application (Frontend on Vercel Edge)**: https://chikkamagaluru-resort-booking.vercel.app
- **Live Backend API (Railway Cloud)**: https://resort-backend-production.up.railway.app

---

1. Executive Summary & Client Context

Company A operates a boutique coffee-estate luxury resort in Chikkamagaluru, Karnataka, nestled in the Western Ghats. The property features **76 keys** comprising **64 Estate Rooms** and **12 Private Pool Villas**, alongside outdoor banquet lawns for destination weddings and corporate offsites. The resort employs 140 staff and generates ₹24–28 crore in annual revenue under the leadership of the General Manager (GM).

1.1 The Business Problem & Financial Opportunity

- **Current State**:

  - 68% of room nights originate from Online Travel Agencies (OTAs) charging 18–22% commission (averaging 20%).

  - Only 12% of bookings occur directly through resort channels because the existing website is an informational brochure with a telephone number.

  - Event enquiries arrive haphazardly across personal WhatsApp, email, and social media, with rates manually maintained in spreadsheets.

  - Approximately 30% of event enquiries wait over 24 hours for a response, forfeiting an estimated 3 events per month.

- **Financial Baseline**:

  - 76 keys × 365 days × 62% average occupancy = 17,199 room nights at an average daily tariff of ₹9,500 = ₹16.34 crore total lodging revenue.

  - OTA commission toll: 68% via agencies = 11,695 room nights = ₹11.11 crore gross bookings. At 20% commission, the resort forfeits **₹2.22 crore annually**.

  - Direct conversion impact: Shifting 15% of agency bookings to direct saves **₹33.3 lakh/year** in commission.

  - Event conversion impact: 3 lost events/month at ₹2 lakh contribution = 36 events × ₹2 lakh = **₹72 lakh/year**.

- **Total annual bottom-line opportunity**: **₹1.053 crore/year**.

- **The Brief's Core Mandate**:

  - Deliver a high-conversion, mobile-responsive direct-booking engine allowing guests to select dates, view real-time availability and dynamic pricing with statutory GST, and complete reservations with zero overbooking risk.

  - Implement a centralized enquiry intake and AI triage pipeline that ingests web form submissions, simulated emails, and simulated WhatsApp messages, classifies intent, extracts key entities, assigns a specialist owner, and prepares grounded response drafts with human-in-the-loop approval.

---

2. Requirements Baseline & Evidence-Based Status

The tracked brief in `docs/isocode-assignment-brief.md` is the requirements baseline used here. The statuses below reflect source review and the checks actually run on 8 November 2026; they are not a blanket certification of compliance, legal compliance, or production readiness.

Requirement Category	Brief Requirement	Evidence-Based Status (8 Nov 2026)	Evidence and Limit
Direct Booking Engine	Photos, availability, rate plans, tax display, test payment, confirmation	**Implemented; verified end-to-end**	Guest routes and client flow are present; frontend tests (25/25) and backend tests pass. Confirmed by one authorized production booking (`FE9JZC9DN7PM`). No additional production bookings or inventory modifications were made.
Allotment Control	Sell admin-set allotment and avoid concurrent overbooking	**Verified against disposable MySQL 8.0.45; merged and deployed**	Under evaluated MySQL 8.0.45 InnoDB scenarios with `SELECT ... FOR UPDATE` row locks, 100 consecutive last-room race iterations, multi-night overlapping ranges, idempotency races, payment/expiry races, and rollback integrity tests passed with zero overselling across the reported 298-test disposable-MySQL run. Merged into `main` at commit `296100f` and deployed.
Admin Controls	Rates, allotment, blackout dates, promo codes	**Implemented; automated tests pass**	Admin services and endpoints are covered by unit/mock tests. No production records were changed.
Enquiry Inbox	Web form, simulated email, and simulated WhatsApp in one inbox	**Implemented; automated tests pass**	Staff inbox and simulated adapters are present. No live enquiry was submitted in this audit.
AI Router & Extraction	Classify, extract fields, assign an owner, and draft a reply	**Implemented; live model behavior not re-verified**	Gemini integration and deterministic fallback exist. This audit did not send a new request to the external AI service.
AI Safety & Fallbacks	Human approval, low-confidence/complaint routing, table-grounded prices	**Implemented; guardrails tested, model output not guaranteed**	Draft sanitization, PII masking, staff review, and routing tests exist. Sanitization is defense-in-depth, not a guarantee against every possible model claim.
Structured Event Form	Structured wedding and event enquiry form	**Implemented; source behavior tested**	`/enquire` provides structured fields and neutral meeting-space selection. No production enquiry was created.
Daily Reservation Export	Daily booking export	**Implemented; endpoint covered by tests**	CSV export is present with formula-injection sanitization. No production data was modified; current export contents were not reviewed here.
Data Privacy & Legal	Handle personal data responsibly; collect Form C details at check-in	**Privacy controls present; legal compliance NOT CERTIFIED**	Consent and private confirmation access are implemented. This is not a legal audit or DPDP compliance certification.
Responsive Design	Minimal, mobile-responsive pages without horizontal overflow	**Partially verified**	Local preview of the landing page at exact 390px and 768px CSS widths showed no horizontal overflow after a header fix. Other routes and 1280px were not measured in this run.

Assumptions and Limits

The financial and occupancy figures in the brief are planning assumptions, not independently measured resort results. Room names, descriptions, capacities, amenity text, and demo tariffs seeded in `V3__reference_data.sql` are project demo data and have not been confirmed by the resort. The displayed 2:00 PM check-in and 11:00 AM checkout times originate from frontend presentation templates as demo assumptions; they have not been confirmed by the resort owner and must not be presented as confirmed resort policy. Tax slabs are explicitly marked `UNVERIFIED` in reference data and must not be represented as legally validated statutory advice. The booking window (`2026-11-02` to `2027-04-30`) is a demo configuration assumption and must be made configurable or rolled forward before bookings beyond that period are needed. Payment and message delivery are simulated. The overbooking prevention objective was evaluated against disposable MySQL 8.0.45 across 100 repeated last-room races and multi-thread integration tests with zero invariant failures; this demonstrates transaction isolation under the tested conditions but does not constitute an absolute guarantee under unmodeled production failures (e.g. cross-region network partitions or connection pool exhaustion). Both the inventory concurrency verification (PR #12) and the AI tax-grounding fix (PR #13) are merged into `main` at commit `296100f` and deployed. One confirmed production booking exists: `FE9JZC9DN7PM` .

---

3. High-Level System Architecture & Technology Choices

The platform uses a modular, three-tier architecture intended to separate guest, staff, and persistence responsibilities. Transactional integrity and availability depend on database behavior and deployment configuration and are not guaranteed by architecture diagrams alone:

```
 [ Edge Client Layer ]
   ├── React 18 SPA (Vite)
   ├── Responsive Design System (CSS Custom Properties, 390px Mobile to 1280px+ Desktop)
   └── Vercel Edge Network (Global CDN, Security Headers, Gzip/Brotli Asset Delivery)
          │
          │ REST / HTTPS (Stateless JSON APIs)
          ▼
[ Application Layer (Railway Cloud Container) ]
   ├── Spring Boot 3.3.5 / Java 21 LTS
   ├── Spring Security 6 (Stateless HMAC-SHA256 JWT, Role & Lane Authorization)
   ├── Transaction Manager (Pessimistic Row-Level Locking, 10-Minute Hold Scheduler)
   ├── Gemini AI Router (configurable Gemini model; code default `gemini-3.8-flash`, structured
   ├── Pricing & provisional tax display engine (seeded tax slabs marked `UNVERIFIED`)
   └── Simulated Delivery Adapters (Email SIM, WhatsApp SIM with Unique Delivery IDs)
          │
          │ JDBC Connection Pool (HikariCP, utf8mb4)
          ▼
[ Persistence Layer (Managed MySQL 8.0 on Railway) ]
   ├── Flyway Database Migrations (V1 to V8 versioned schema, reference data, and draft correcti
   ├── 12 Normalized Relational Tables with Foreign Key Constraints
   └── Inventory row locks and counters (`daily_inventory_rates.booked_count`)
```

Why These Technologies Were Chosen

1. **Java 21 LTS & Spring Boot 3.3.5**:

- **Concurrency & ACID Transactions**: Inventory holds use Spring transactions and database row locks. Java 21 virtual-thread capability is not itself a concurrency-safety mechanism; database-level pessimistic locks

(`SELECT ... FOR UPDATE` in ascending date order) within `@Transactional(isolation = Isolation.READ_COMMITTED)` provide the isolation mechanism, verified on MySQL 8.0.45 across 100 repeated race iterations with zero overselling (merged into `main` at `296100f`).

- **Type Safety**: Financial calculations (room tariffs, GST percentages, promo deductions) are strictly bounded by `BigDecimal`, preventing floating-point inaccuracies.

2. **React 18 & Vite**:

- **Build Output**: The 8 November 2026 local production build emitted a 203.77 KB main JavaScript chunk (66.05 KB gzip). This does not include all lazy chunks, images, or network transfer, and no 3G/4G performance measurement was run.

- **Component Decoupling**: Guest checkout and staff operational dashboards are split into route-level components; render performance was not profiled.

3. **Managed MySQL 8.0**:

- **Functional Unique Indexes**: The V6 migration adds a functional unique index using `COALESCE(room_category_id, 0)` for resort-wide and category-specific blackout dates. The MySQL migration/concurrency path was not exercised in this audit run.

- **Pessimistic Locking**: `SELECT ... FOR UPDATE` is used in the inventory transaction path. Evaluated against disposable MySQL 8.0.45 across the reported 298-test suite and merged into `main` at `296100f`.

4. **Gemini API (configured model)**:

- **Structured Extraction**: The configured model is asked for structured enquiry fields and a confidence value, then application logic validates and routes the result. The current code default is `gemini-3.8-flash`; the deployed environment's selected model and live response behavior were not independently checked in this audit.

---

4. Relational Data Model & Database Schema

The V2 Flyway migration creates 12 tables. Later migrations add enquiry fields, reference data, tax configuration, a blackout uniqueness index, and corrections to demo drafts. The schema does **not** contain a `booking_allocations` table; booking activity is reflected in `daily_inventory_rates.booked_count`.

The actual V2 tables are:

1. `users`
2. `room_categories`
3. `physical_units`
4. `rate_plans`
5. `daily_inventory_rates` (base rate, website allotment, booked count, and optimistic version)
6. `blackout_dates`
7. `promo_codes`
8. `gst_tax_slabs`

9. `bookings`
10. `payments`
11. `enquiries`
12. `enquiry_drafts`

`physical_units`, `rate_plans`, `daily_inventory_rates`, and `blackout_dates` reference `room_categories`. `bookings` references a room category, rate plan, and optional promo code. `payments` has a one-to-one foreign-key relationship with `bookings`. Enquiries may be assigned to a user, and `enquiry_drafts` references an enquiry and the staff member who sends a draft.

Key Entity Specifications

1. **`room_categories` and `physical_units`**: The brief describes a 76-key property; the reference seed divides demo inventory into 64 room units and 12 villa units. This category split and related marketing copy are project data, not separately confirmed property facts.

2. **`daily_inventory_rates`**: Stores `base_rate_inr`, `website_allotment_total`, and `booked_count` for each room category and stay date. The booking service updates these locked rows within its transaction.

3. **`bookings`**: The schema status values are `PENDING_PAYMENT`, `CONFIRMED`, `FAILED`, `EXPIRED`, and `CANCELLED`. It stores the booking's pricing snapshot, guest data, dates, and `expires_at` timestamp.

4. **`payments`**: Stores a unique single-use token and mock payment state for each booking; the gateway is simulated.

5. **`enquiries` and `enquiry_drafts`**: Store incoming messages, classification, confidence, assignment, and a response draft that requires staff review.

6. **`blackout_dates`**: V6 adds the MySQL functional unique index `uk_blackout_dates_scope_date` on `((COALESCE(room_category_id, 0)), blackout_date)` to prevent duplicate resort-wide or category-specific closures for a date.

---

5. Core System Workflows & Business Logic

5.1 Three-Stage Guest Booking Funnel (Inventory-Control Design)

The guest booking journey is designed around three clear steps. The inventory transaction protects the admin-set website allotment under concurrent requests, as verified in disposable MySQL 8.0.45 integration tests and merged into `main` at `296100f` and deployed.

```
[ Step 1: Details & Consent ] ──▶ [ Step 2: Payment Simulation ] ──▶ [ Step 3: Instant Confirm
       (/checkout)                       (/payment/:ref)                    (/bookings/:ref)
  - Guest Name, Email, Phone        - Authoritative UTC Expiry Clock    - Reference Code (BKG-X
  - Privacy Notice & Consent        - 10-Minute Inventory Hold           - Detailed Price & pro
  - Promo Code Validation           - Idempotent Mock Payment            - Private Confirmation
```

1. **Step 1: Guest Details & Consent (`/checkout`)**:

- The guest selects their preferred room and rate plan (`EP` Room Only, `CP` Breakfast Included, `MAP` Breakfast + Dinner).

- Validates guest full name, email, and mobile number.

- Requires the guest to acknowledge the privacy notice and consent checkbox; this implementation is not a legal compliance certification.

- Upon form submission, the backend executes an atomic transaction in `BookingTransactionService`:

- Checks resort and category blackout dates.

- Performs `SELECT ... FOR UPDATE` row locks across each requested stay date in `daily_inventory_rates`.

- Verifies `booked_count + requested_rooms <= website_allotment_total`.

- Increments `booked_count` and generates a booking record with `status = 'PENDING_PAYMENT'`; the backend clock sets `expires_at` ten minutes after hold creation.

2. **Step 2: Payment Simulation (`/payment/:ref`)**:

- Displays live reservation summary and active hold countdown timer.

- **Timezone-Safe Architecture**: The backend serializes `expires_at` as an explicit UTC ISO-8601 string ending in `Z` (e.g. `2026-11-08T04:15:30Z`), preventing browser timezone shifts (such as India Standard Time UTC+5:30) from causing premature client expiration.

- The client timer serves purely as visual feedback; the backend remains the authoritative validator. Submitting payment verifies hold validity before confirming.

- Payment is processed idempotently using a UUID request key.

3. **Step 3: Instant Confirmation (`/bookings/:ref`)**:

- Updates booking status to `CONFIRMED`.

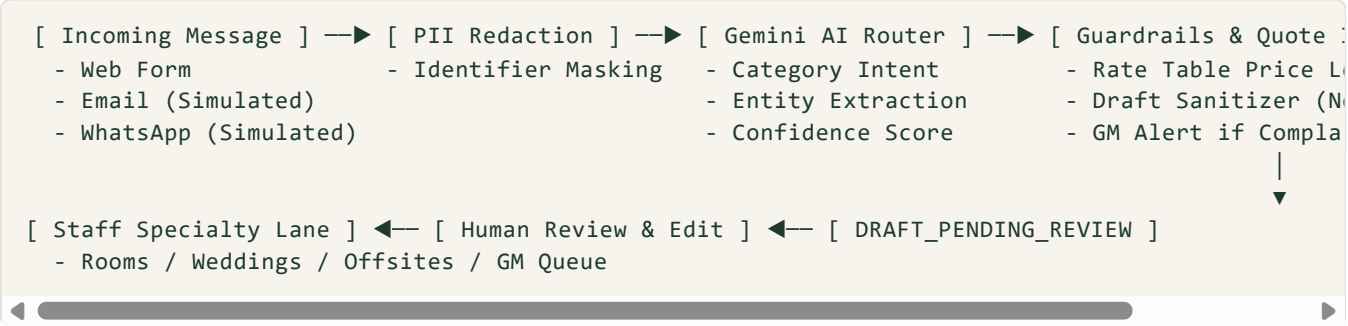
- Generates a human-readable booking reference (`BKG-XXXXXX`).

- Displays itemized room rate breakdown, promotional discount, and provisional GST.

- Does not expose public search lookups; booking access is private via confirmation link.

5.2 Multi-Channel Enquiry Intake & AI Triage Pipeline

Enquiries from the public web form (`/enquire`), forwarded emails, and WhatsApp arrive in a unified triage queue:



1. **PII Masking & Safe Prompting**:

- The raw enquiry message is passed through a redactor that masks contact and common government-ID patterns before Gemini dispatch. This is a privacy control, not proof of legal compliance; the original enquiry remains in the staff system.

2. **Gemini Classification & Extraction** (configurable; code default `gemini-3.8-flash`):

- Categorizes enquiry into `ROOM_BOOKING`, `WEDDING`, `CORPORATE_OFFSITE`, `COMPLAINT`, or `UNSORTED`.

- Extracts structured parameters: `checkInDate`, `checkOutDate`, `guestCount`, `roomCount`, `approximateBudget`.

- Generates a contextual reply draft and computes a classification confidence score between `0.00` and `1.00`.

3. **Draft Safety Guardrails (Defense in Depth)**:

- **Quote Injection (`QuoteInjectorService`)**: When dates and room categories are identified, the application looks up rates from `daily_inventory_rates` and injects the quote; the AI is not the source of prices.

- **Draft Sanitizer (`DraftPriceSanitizer`)**: Removes unsupported numeric price claims and defers unverified facility/amenity claims (including AV and room-hold promises). This reduces, but cannot eliminate, model-output risk.

- **Safety Rules**:

- Enquiries with confidence below the configured threshold (default `0.80`) are routed to the **Unsorted Lane** monitored by the General Manager.

- Enquiries classified as `COMPLAINT` immediately trigger a `gm_alerted = true` flag.

- **Drafts are NEVER sent automatically**: Every draft defaults to `DRAFT_PENDING_REVIEW` and requires reservations staff or GM review, editing, and one-click dispatch approval.

- Staff can re-route any misclassified enquiry with a single click.

---

6. UI & Design Decisions: The 7 Core Screens

The design language embodies the client brief's requirement: "*The client values good, minimal design: clean layouts, white space, a restrained palette and clear type, with nothing on screen that lacks a purpose.*"

The color palette reflects the plantation landscape: **Deep Forest Green** (`#1E3A2F`), **Warm Sand Cream** (`#F7F4EE`), **Coffee Muted Slate** (`#5C5852`), and **Border Cream** (`#E5E0D8`).

Screen 1: The Landing Page (`/`)

- **Design Intent**: Sells the estate retreat experience without overwhelming the guest with competing actions.
- **Booking Controls**:

  - The hero section presents WebP photography with warm typography. `Book a Stay` scrolls/focuses the date form; `Plan an Event` links to the enquiry flow.

  - The search form's `Search Availability` button validates selected dates and submits the search. The header's `Check Availability` shortcut also scrolls to this form when displayed; it is hidden on booking subroutes and at widths up to 480px.

Screen 2: Stay / Availability & Rooms (`/stay`)

- **Design Intent**: Instant clarity on room options and rate plans with zero friction.
- **Single Compact Filter Bar**: Features a single horizontal bar (`Check-in | Check-out | Guests | Modify Search`).

- **Date-First State**: No stay dates are silently prefilled. A missing or invalid date pair shows a date-entry form and prompt; it does not call availability until valid dates are selected and submitted. A valid URL date pair loads availability automatically.
- **Navbar Deduplication**: The navbar on `/stay` omits the generic "Check Availability" button, showing only a clean `← Back to Estate` utility link.
- **Robust Empty State**: If a date is fully booked, displays an informative card with formatted dates (never broken `"for to is"`), with actions to adjust dates or submit a direct enquiry.

Screen 3: Room Details (`/rooms/:code`)

- **Design Intent**: Immersive room exploration with immediate booking capability.
- **Preserved Stay Criteria**: Displays the selected stay dates, night count, and guest count in a clean summary strip with a prominent `Change dates` link that returns to `/stay` with all criteria preserved.
- **Dedicated Primary CTA**: Rate plan cards display clear inclusions (`EP` , `CP` , `MAP`) and a single primary button: **`Book Now`** .

Screen 4: Three-Step Checkout (`/checkout` ➔ `/payment/:ref` ➔ `/bookings/:ref`)

- **Design Intent**: Distraction-free, linear conversion funnel.
- **Layout**: Two-column desktop layout (guest form on the left, sticky booking summary card on the right).
- **Primary Actions**:

  - Step 1 (`/checkout`): Single primary button **`Proceed to Payment`** .

  - Step 2 (`/payment/:ref`): Clear 10-minute countdown indicator and primary button **`Simulate & Confirm Reservation`** .

  - Step 3 (`/bookings/:ref`): Complete confirmation breakdown with reference code and printable receipt.

Screen 5: Staff Enquiry Inbox (`/staff/enquiries`)

- **Design Intent**: High-velocity operational triage hub for the reservations team and GM.
- **Layout**:

  - Left panel: Filterable specialty tabs (`All` , `Rooms` , `Weddings` , `Offsites` , `Complaints` , `Unsorted`).

  - Enquiry cards display channel icon (Web, Email, WhatsApp), sender, confidence score badge, and GM alert indicator.

  - Right detail panel: Displays the raw customer message, AI extracted parameters, 1-click re-route dropdown, editable draft response, and one-click dispatch button.

Screen 6: Staff Rate & Allotment Admin (`/staff/inventory`)

- **Design Intent**: Comprehensive inventory management preventing overbooking.
- **Capabilities**:

  - Interactive calendar grid allowing staff to view and modify daily base rates, total website allotment, and booked counts across categories.

  - Blackout date management for resort-wide or category-specific closures.

  - Promotional discount code manager (active dates, discount percentages, usage tracking).

  - One-click **Daily Booking CSV Export** sanitized against spreadsheet injection (CWE-1236).

Screen 7: Structured Event & Wedding Form (`/enquire`)

- **Design Intent**: Guided multi-parameter intake for complex destination events.
- **Capabilities**:

  - Interactive category selector (`Weddings` , `Corporate Offsites` , `Individual Stays`).

  - Contextual fields for estimated guest counts, preferred dates, room blocks, catering preferences, budget estimates, and conference setup needs.

  - Neutral default choices (e.g. conference hall selection defaults to "Not specified / please advise").

  - Single primary CTA: `Submit Enquiry` .

---

7. Scope Cuts & Future Roadmap

In strict accordance with the client brief's *CORE BUILD* guidelines and *OUT OF SCOPE* boundaries:

What Was Cut (and Why)

1. **Channel Manager / OTA Two-Way Sync**: Explicitly declared out of scope by the brief. The resort controls website overbooking by allocating an admin-set website allotment (`website_allotment_total`) that acts as an isolated inventory pool.

2. **Production Payment Gateway**: Replaced with an internal deterministic test payment simulator (`POST /api/v1/public/payments/{ref}/mock-webhook`), eliminating non-commercial financial transaction compliance while validating the complete state machine.

3. **Loyalty Program & Restaurant Reservations**: Excluded to focus development velocity exclusively on the core direct-booking engine and AI triage workflows.

4. **Outbound Third-Party Telephony/SMS Rails**: Replaced with internal simulated delivery adapters (`EMAIL_SIM` , `WHATSAPP_SIM`) generating verified delivery IDs.

What to Do Next (Production Roadmap)

1. **Production Payment Gateway Integration**: Connect Razorpay / Stripe Webhooks with UPI intent flow for Indian domestic travelers.

2. **PMS Webhook Integration**: Implement bidirectional synchronization with hotel property management systems (e.g. Opera, Cloudbeds) for automated daily reservation synchronization.

3. **WhatsApp Business Cloud API**: Transition simulated WhatsApp ingestion to official Meta Cloud API webhooks with end-to-end template verification.

4. **GM Financial Dashboard**: Implement a real-time analytics view tracking saved OTA commission metrics (moving towards the target ₹33.3 lakh/year saving).

---

8. Responsive Viewport Checks (8 November 2026)

The built local preview was rendered in same-origin browser frames with exact CSS viewport widths. Landing, date-first Stay, corporate-offsite enquiry, and staff login routes were measured; this does not certify every guest or staff route or a physical device.

Viewport (`window.innerWidth`)	Routes tested	Document width / `scrollWidth`	Result
390px	`/`, `/stay`, `/enquire?type=offsite`, `/staff/login`	375px / 375px on all four	**PASS; no horizontal overflow**
768px	`/`, `/stay`, `/enquire?type=offsite`, `/staff/login`	753px / 753px on the first three; 768px / 768px on staff login	**PASS; no horizontal overflow**

At 390px, the initial landing measurement found `.guest-header__cta` extending beyond the page edge; hiding that duplicate header action at widths up to 480px removed the overflow. The mobile navigation toggle was also exercised: it opened with `aria-expanded="true"`, exposed four links, and closed with `aria-expanded="false"`. Other routes, 1280px layout, and all tap-target dimensions still need explicit checks; the 44px CSS token does not substitute for measuring every control.

9. Release, Audit, and Concurrency Evidence Notes (8–9 November 2026)

The repository `main` baseline is commit `296100f`, which incorporates both PR #12 (hardened concurrency verification) and PR #13 (AI tax-grounding on unverified GST). Both Vercel and Railway deployments are verified as running artifacts built from commit `296100f`:

- **Vercel Frontend**: Successfully deployed commit `296100f` (`https://chikkamagaluru-resort-booking.vercel.app/`).
- **Railway Backend**: Running deployment `62741d27-4271-43d2-bc95-0145165c1bd0` built from `296100f`, verified via container logs at 2026-11-09T04:48:40.424Z (10:18:40 AM IST), executing Flyway schema v8 and Tomcat on port 8080 (`https://resort-backend-production.up.railway.app`).
- **Confirmed Production Booking**: One live confirmed production booking exists: reference `FE9JZC9DN7PM`. (Guest contact phone number is omitted for privacy). In accordance with pre-submission safeguards, no additional production bookings or inventory writes were performed, and existing reservations were not cancelled or altered.
- **Check-in / Check-out Presentation Policy**: The displayed 2:00 PM check-in and 11:00 AM checkout times on `frontend/src/pages/BookingConfirmationPage.jsx` and `Styleguide.jsx` originate from frontend UI presentation templates as **demo assumptions**. They have not been confirmed by the resort owner and must not be presented as confirmed resort policy. No production values were modified.

Concurrency Verification on Disposable MySQL 8.0.45 (9 November 2026)

- **Branch**: Merged into `main` at commit `296100f` and deployed.
- **Environment**: Local MySQL Server 8.0.45 on Win64 (disposable test database `disposable_concurrency_repeat_test_db`).
- **Hardened Disposable Database Guard (`DisposableDatabaseGuard`)**:

  - *What the guard does*:

  1. Parses the JDBC connection URL to isolate scheme, host authority, database name, and parameters.

  2. Enforces an explicit host policy: only local loopback targets (`localhost`, `127.0.0.1`, `::1`, `[::1]`) are allowed. All remote, shared, or production hostnames/IPs (e.g. `production.railway.internal`, `aws-rds`,

`shared-staging.internal`, `192.168.x.x`) are rejected by default, even if the database name contains `test` or `disposable`.

3. Enforces an explicit database naming policy: requires names matching `disposable_`, `test_`, `_test`, or `_test_db`, while rejecting system databases (`mysql`, `information_schema`, `performance_schema`, `sys`) and any database name containing `prod`, `live`, `main`, or `shared`.

4. Enforces zero credential or connection-string leakage: exception messages are strictly static and generic; no connection string, host, database name, or credentials appear in messages or logs.

5. Covered by 38 automated test cases in `DisposableDatabaseGuardTest` verifying allowed disposable configurations, host rejection, database name rejection, malformed URLs, and zero leakage.

- *What the guard does NOT guarantee*:

1. It cannot detect OS-level network remapping or port redirection, such as SSH local port tunnels (e.g. `ssh -L 3306:remote-prod:3306`) or malicious `/etc/hosts` / `hosts` file aliasing mapping `localhost` to an external host IP.

2. It does not inspect the contents of a local database; if production data were dumped into a local database named `disposable_test_db`, the guard would not detect data provenance.

- **Full Backend Suite Result**: All **298 tests across 29 test classes passed** (0 failures, 0 errors, 0 skipped):

- `AllotmentConcurrencyIntegrationTest`: 9 tests passed, including 100 consecutive repeated runs of competing requests for the last available room (100 passed, 0 failures), multi-night overlapping ranges (10 threads 14–16 Nov vs 10 threads 15–17 Nov), idempotency races, payment vs expiry races, and rollback integrity tests asserting `booked_count` remains strictly unchanged after rejected/failed transactions.

- `BlackoutConcurrencyIntegrationTest`: 4 tests passed.

- `DatabaseMigrationIntegrationTest`: 5 tests passed (all 12 tables, reference data, GST slabs, DB check constraints, AI fields).

- `DraftApprovalConcurrencyIntegrationTest`: 2 tests passed.

- `V8MigrationIntegrationTest`: 1 test passed.

- `DemoDataSeederIntegrationTest`: 1 test passed.

- `DisposableDatabaseGuardTest`: 38 tests passed.

- Remaining unit/mock suites: 238 tests passed.

- **Teardown Proof**: Disposable database `disposable_concurrency_repeat_test_db` dropped and confirmed absent from `SHOW DATABASES` and `INFORMATION_SCHEMA.SCHEMATA`.
- **Scope Limit**: Under evaluated MySQL 8.0.45 InnoDB scenarios with `SELECT ... FOR UPDATE` in ascending date order, no overselling occurred; this demonstrates row-level isolation in the evaluated test environment and does not guarantee behavior under network partitions or connection pool exhaustion.